Database Normalization

A Complete Guide to Normal Forms

1NF **2NF** **3NF** **BCNF** **4NF** **5NF**

Covering: Anomalies • Dependencies • Decomposition • Examples

📋 Agenda — What We'll Cover

01 **Why Normalization?**

Problems in unnormalized databases

02 **Functional Dependencies**

The foundation of normal forms

03 **1NF**

First Normal Form — Atomicity

04 **2NF**

Second Normal Form — Full Dependency

05 **3NF**

Third Normal Form — Transitive Dependency

06 **BCNF**

Boyce-Codd Normal Form

07 **4NF**

Fourth Normal Form — Multi-Valued Dependencies

08 **5NF**

Fifth Normal Form — Join Dependencies

What is Database Normalization?

Database Normalization is the process of organizing a relational database to reduce data redundancy and improve data integrity by decomposing tables into smaller, well-structured relations following a set of formal rules called Normal Forms.

Proposed by: E.F. Codd in 1970 as part of his relational model theory

Goal: Each piece of data stored in exactly one place — no redundancy

Method: Apply progressively stricter Normal Forms (1NF → 5NF)

Result: Cleaner schema, fewer anomalies, easier maintenance

⚠️ Problems Without Normalization

StudentID	StudentName	CourseID	CourseName	InstructorID	InstructorPhone
S01	Alice	C101	Math	I01	555-1234
S01	Alice	C102	Physics	I02	555-5678
S02	Bob	C101	Math	I01	555-1234

Insertion Anomaly

Cannot add a new course without enrolling a student first. Course data is tied to student enrollment.

Update Anomaly

Changing Instructor I01's phone requires updating multiple rows. Miss one → inconsistency.

Deletion Anomaly

Deleting the last student in a course (e.g., Bob drops Math) deletes course & instructor data too.

Redundancy

Alice's name appears twice. I01's phone stored in every Math enrollment — wastes space.

Benefits of Normalization

Eliminates Redundancy

Each fact stored once — no duplicate data across rows or tables.

Ensures Data Integrity

Updates apply in one place, preventing inconsistent states.

Faster Updates

Smaller tables mean quicker INSERT, UPDATE, DELETE operations.

Easier Queries

Well-structured tables make SQL queries more intuitive to write.

Flexible Schema

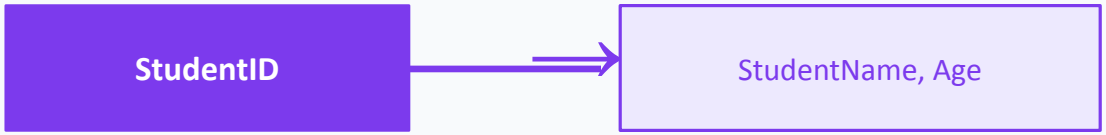
Adding new entity types requires fewer structural changes.

Less Storage

Avoiding repeated data reduces disk usage significantly.

Functional Dependencies — The Foundation

A functional dependency X ⟶ Y means: for every value of X, there is exactly one value of Y. X functionally determines Y.



Full FD

Y depends on the entire composite key, not a part of it.
Ex: (StudentID, CourseID) ⟶ Grade

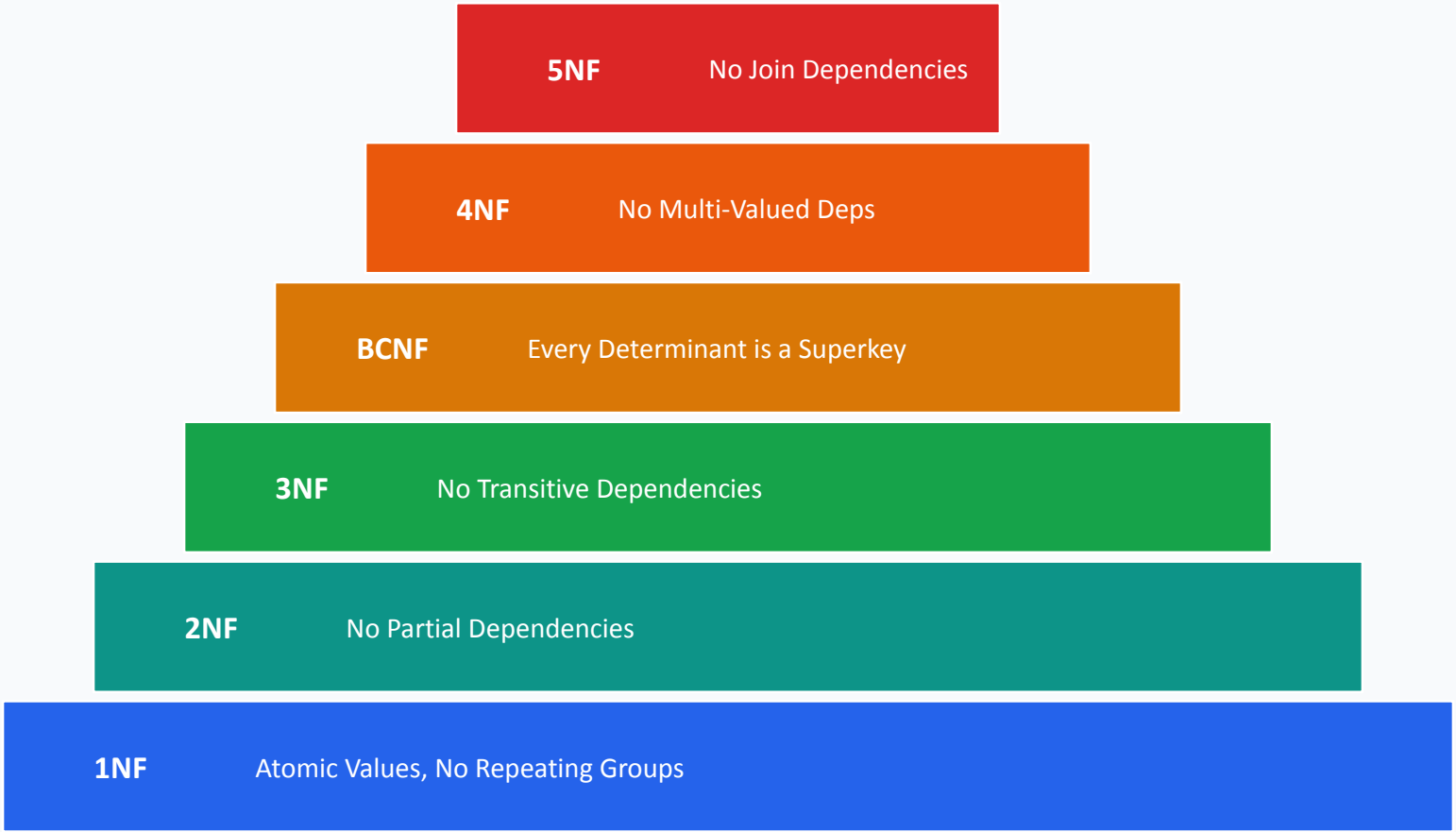
Partial FD

Y depends on only part of a composite key.
Ex: (StudentID, CourseID) ⟶ StudentName [only on StudentID]

Transitive FD

X ⟶ Y ⟶ Z (Y is non-key, Z is non-key).
Ex: StudentID ⟶ DeptID ⟶ DeptName

The Normalization Hierarchy



First Normal Form (1NF)

Atomicity • Single-Valued Attributes • No Repeating Groups

1NF — Rules & Requirements

1. **Atomic Values:** Every column must contain only indivisible (atomic) values. No lists, arrays, or sets in a single cell.
2. **Same Data Type:** All values in a column must be of the same domain/type (e.g., all integers or all strings).
3. **Unique Column Names:** Each attribute must have a unique name within the table.
4. **No Repeating Groups:** Eliminate repeating columns like Phone1, Phone2, Phone3 — use a separate table.
5. **Primary Key:** Each row must be uniquely identifiable by a primary key.

1NF — Before & After Example

Violates 1NF (Non-Atomic Values)

StudentID	Name	Courses (Multi-valued!)
S01	Alice	Math, Physics, CS
S02	Bob	Math, Chemistry
S03	Carol	Physics

Issues: 'Courses' column stores a list — violates atomicity. Cannot query individual course easily.

→

Satisfies 1NF (Atomic Values)

StudentID	Name	Course
S01	Alice	Math
S01	Alice	Physics
S01	Alice	CS
S02	Bob	Math
S02	Bob	Chemistry
S03	Carol	Physics

Each cell holds one value. PK = (StudentID, Course). Fully atomic — satisfies 1NF.

Second Normal Form (2NF)

Full Functional Dependency • No Partial Dependencies

2NF — Rules & Requirements

Prerequisites: The table must already be in 1NF.
Definition:
Every non-prime attribute must be fully functionally dependent on the entire primary key (no partial dependencies).

Partial Dependency = A non-key attribute depends on only part of a composite primary key.

Example Scenario — Order Details Table

OrderID	ProductID	Quantity	ProductName	UnitPrice
O01	P10	5	Laptop	$999
O01	P20	2	Mouse	$25
O02	P10	1	Laptop	$999

🔑 *Primary Key: (OrderID, ProductID)* | ⚠️ *Partial FDs: ProductName → ProductID only; UnitPrice → ProductID only*

ProductName and UnitPrice depend only on ProductID, not on the full (OrderID, ProductID) key — this is a Partial Dependency, which violates 2NF!

2NF — Decomposition Example

Fix: Decompose into two tables — remove partial dependencies

Table 1: OrderDetails (in 2NF)

OrderID 🔑	ProductID 🔑	Quantity
O01	P10	5
O01	P20	2
O02	P10	1

Quantity depends on BOTH OrderID and ProductID (full FD)

Table 2: Products (in 2NF)

ProductID 🔑	ProductName	UnitPrice
P10	Laptop	$999
P20	Mouse	$25

ProductName and UnitPrice depend only on ProductID

Benefits of 2NF Decomposition:

- ✓ No update anomaly: Change Laptop price in ONE row in Products only
- ✓ No insertion anomaly: Add a product without creating an order
- ✓ No deletion anomaly: Delete all orders for P10 — product record stays in Products

Third Normal Form (3NF)

No Transitive Dependencies • Direct Key Relationships

3NF — Rules & Example

Prerequisite: Must be in 2NF.
Rule:
No non-prime attribute should be transitively dependent on the primary key (X → Y → Z, where Y is non-prime).

Violates 3NF — Transitive Dependency

EmpID 🔑	EmpName	DeptID	DeptName
E01	Alice	D10	Engineering
E02	Bob	D20	Marketing
E03	Carol	D10	Engineering

EmpID → DeptID → DeptName (Transitive!)
DeptName depends on DeptID, not EmpID

Decomposed to 3NF

EmpID 🔑	EmpName	DeptID
E01	Alice	D10
E02	Bob	D20
E03	Carol	D10

Employees table: DeptName removed

DeptID 🔑	DeptName
D10	Engineering
D20	Marketing

Departments table: DeptName depends directly on DeptID

2NF vs 3NF — Key Differences

Aspect	2NF	3NF
Eliminates	Partial dependencies	Transitive dependencies
Dependency Type	Non-key attr on part of PK	Non-key attr on another non-key attr
PK Type	Applies to composite keys	Applies to any key
Example FD Removed	(OrderID,ProdID)→ProdName via ProdID only	EmpID→DeptID→DeptName via non-prime DeptID
Still Possible Issue	Transitive deps may exist	Superkey violations may exist (BCNF)

💡 **Codd's Theorem:**

A relation in 3NF can always be decomposed losslessly and dependency-preservingly. This makes 3NF the minimum practical normalization target for most production databases — it eliminates both partial and transitive dependencies.

Boyce-Codd Normal Form (BCNF)

Stronger version of 3NF • Every Determinant is a Candidate Key

BCNF — Rules & Example

Prerequisite: Must be in 3NF.
Rule:
For every non-trivial functional dependency X ⟶ Y, X must be a superkey. BCNF is stricter than 3NF — a table in 3NF may not be in BCNF when there are multiple overlapping candidate keys.

Example: Course-Teacher-Room Scheduling

Student	Subject	Teacher
Alice	Math	Prof. Jones
Bob	Math	Prof. Jones
Alice	Physics	Prof. Smith
Bob	Physics	Prof. Smith

Super Keys: (Student,Subject) and (Student,Teacher)
FD: Teacher ⟶ Subject (Teacher determines Subject)
But Teacher is NOT a superkey ⟶ BCNF Violation!

BCNF Decomposition

Teacher 🔑	Subject
Prof. Smith	Math
Prof. Jones	Physics

Student 🔑	Teacher
Alice	Prof. Smith
Bob	Prof. Jones

Each determinant is now a candidate key Teacher⟶Subject is in its own table.

3NF vs BCNF — When They Differ

Aspect	3NF	BCNF
Rule	No transitive deps on Primary Key	Every determinant is a superkey
Strictness	Less strict	Stricter
Lossless Decomp	Always possible	Always possible
Dependency Preservation	Always possible	Not always possible
Handles overlapping CKs?	Partially	Fully
Practical Use	Most OLTP systems	When anomalies from overlapping CKs exist

Trade-off:

BCNF may not always preserve all functional dependencies after decomposition. In those cases, 3NF is preferred to maintain both lossless joins AND dependency preservation. Choose BCNF when anomaly elimination outweighs dependency preservation requirements.

Fourth Normal Form (4NF)

No Multi-Valued Dependencies • Beyond BCNF

4NF — Multi-Valued Dependencies

Multi-Valued Dependency (MVD) X →→ Y: For each value of X, there is a set of Y values independent of other attributes. X multi-determines Y.

4NF Rule:

Must be in BCNF and have no non-trivial multi-valued dependencies where X is not a superkey.

Example: Employee Skills & Languages Table

EmployeeID	Skill	Language
E01	Python	English
E01	Python	Spanish
E01	Java	English
E01	Java	Spanish
E02	SQL	English

MVDs present:
E01 → Skill (independent of Language)
E01 → Language (independent of Skill)
This creates redundant elements — if E01 knows Python AND Spanish, must list ALL combinations!

4NF Decomposition

EmployeeID 🔑	Skill
E01	Python
E01	Java
E02	SQL

EmployeeID 🔑	Language
E01	English
E01	Spanish
E02	English

Each MVD separated into its own table
No redundant row combinations needed.

4NF — Real-World Application

Real-World Scenario: A course can have multiple textbooks AND be taught by multiple teachers — independently.

4NF Violation Table

Course	Teacher	Textbook
Math	Smith	Calculus A
Math	Smith	Calculus B
Math	Jones	Calculus A
Math	Jones	Calculus B

Course →→ Teacher AND Course →→ Textbook independently
Storing all combinations = redundant data!

After 4NF Decomposition

Course 🔑	Teacher
Math	Smith
Math	Jones

Course 🔑	Textbook
Math	Calculus A
Math	Calculus B

Only 2+2=4 rows needed instead of 2×2=4 (here same, but scales better: 5 teachers × 10 books = 50 vs 15 rows)

4NF ensures: Independent multi-valued facts about an entity are stored in separate tables, preventing artificial row multiplication.

Fifth Normal Form (5NF)

Join Dependencies • Project-Join Normal Form (PJNF)

5NF — Join Dependencies

Join Dependency: A table T has a join dependency ⋈{T1,T2,...Tn} if T can be reconstructed by joining its projections T1,T2,...Tn without any spurious tuples.

5NF Rule:

Must be in 4NF. Every join dependency in the table must be implied by the candidate keys. A table cannot be decomposed into smaller projections without loss of information.

Example: Supplier-Product-Project Ternary Relationship

Supplier	Product	Project
S1	P1	Proj1
S1	P2	Proj2
S2	P1	Proj2
S1	P1	Proj2

Business Rule:
- S1 supplies P1
- P1 is used in Proj1
- S1 supplies to Proj1

→ Then S1 supplies P1 for Proj1

This creates a join dependency!

5NF — Three binary projections:

Supplier	Product
S1	P1
S1	P2
S2	P1

+ Supplier-Project & Product-Project tables

Key Insight: 5NF handles cases where a ternary relationship CANNOT be replaced by binary relationships without generating spurious tuples. Joining the binary projections back should reconstruct the original table perfectly.

5NF — Detailed Decomposition Example

Original Ternary Table

Agent 🔑	Company 🔑	Product 🔑
A1	C1	P1
A1	C2	P2
A2	C1	P2
A2	C2	P1

5NF Decomposition into 3 binary tables:

Agent	Company
A1	C1
A1	C2
A2	C1
A2	C2

Agent-Company

Agent	Product
A1	P1
A1	P2
A2	P2
A2	P1

Agent-Product

Company	Product
C1	P1
C2	P2
C1	P2
C2	P1

Company-Product

Normal Forms — Complete Summary

NF	Prerequisite	Eliminates	Key Rule	Practical Use
1NF	None	Non-atomic values Repeating groups	Every cell holds one atomic value	Always required
2NF	1NF	Partial dependencies	Non-prime attrs fully depend on entire PK	Composite key tables
3NF	2NF	Transitive deps	No non-prime attr depends on another non-prime	Most OLTP DBs
BCNF	3NF	Superkey violations	Every determinant is a candidate key	Overlapping candidate keys
4NF	BCNF	Multi-valued deps	No non-trivial MVD unless X is superkey	Independent multi-facts
5NF	4NF	Join dependencies	Every join dep implied by candidate keys	Complex ternary/n-ary rels

⚖️ When to Stop Normalizing — Denormalization

Higher normal forms reduce redundancy but can hurt read performance. Denormalization intentionally introduces controlled redundancy for speed. Always profile before deciding.

Normalize to 3NF/BCNF for...

- ✓ OLTP systems (banking, e-commerce orders)
- ✓ Data with frequent INSERT/UPDATE/DELETE
- ✓ Systems where integrity is critical
- ✓ When storage is a constraint

Consider Denormalization for...

- ✓ OLAP / Data Warehouses (star schema)
- ✓ Read-heavy reporting dashboards
- ✓ When JOIN cost is prohibitive on large tables
- ✓ Caching frequently accessed computed values

💡 **Rule of Thumb: Normalize first, then denormalize only where profiling shows a real bottleneck.**

🔄 Step-by-Step: Unnormalized → 3NF

Start: Unnormalized Orders Table

OrderID	Customer	CustCity	Products (list)	Total
O01	Alice	NYC	Laptop, Mouse	$1024
O02	Bob	LA	Keyboard	$80

→1NF Separate multi-valued Products into individual rows. PK=(OrderID,ProductID)

→2NF Move CustCity to Customers table (depends on CustomerID, not full PK). Products table separate.

→3NF Ensure no transitive deps remain. All non-key attrs depend directly on PK only.

Final 3NF Tables: Customers(CustID,Name,City) | Orders(OrderID,CustID,Total) | Products(ProdID,Name,Price) | OrderItems(OrderID,ProdID,Qty)

📌 Quick Reference Cheatsheet

1NF	Atomic values + Unique cols + No repeating groups
2NF	1NF + No partial FD (non-key attr depends on part of composite PK)
3NF	2NF + No transitive FD (non-key attr depends on another non-key attr)
BCNF	3NF + Every determinant X in X⟶Y is a superkey/candidate key
4NF	BCNF + No non-trivial multi-valued dependency (X ⟶⟶ Y) unless X is superkey
5NF	4NF + Every join dependency implied by candidate keys (lossless decomp only)

Memory Aid: '1 Atom, 2 Full, 3 No Transit, BC Superkey, 4 No MVD, 5 No Join Dep'

Key Takeaways

🎯 Normalize Progressively

Apply 1NF first, then 2NF, 3NF, BCNF — each builds on the last.

🔍 Understand Dependencies

Functional, partial, transitive, multi-valued, and join dependencies each target a specific normal form.

⚖️ Balance with Performance

3NF/BCNF is the sweet spot for most systems. Denormalize only when profiling demands it.

📐 Use Real Examples

Apply normalization to your own schema — identify anomalies and decompose step-by-step.